MODULE 3

COMMUNICATION PROTOCOL

A **communication protocol** is a system that allows two or more entities to talk to each other in a communications system. Communication protocols describe communication **rules, syntax** and how the device can recover from errors. Like human beings, communication protocols allow data transmission between hardware, software or a combination of the two with different languages.

Serial Communication

Serial communication is a method of transmitting data **one bit at a time** over a communication channel between devices. In Arduino systems, serial communication is commonly used to exchange data between the **Arduino board and a computer** through the USB connection.

It is mainly used for:

- Sending sensor data to the computer
- Debugging programs
- Controlling Arduino using commands from a computer
- Communication between microcontrollers

In Arduino, serial communication is implemented through the **UART (Universal Asynchronous Receiver Transmitter)** hardware present inside the microcontroller.

Serial Data Communication between Serial Monitor and Arduino

The **Serial Monitor** is a tool in the Arduino IDE that allows communication between the **computer and the Arduino board** through the USB cable.

Communication Process

1. Arduino sends data to the computer using **Serial.print( ) or Serial.println( )**.
2. The computer receives and displays the data in the **Serial Monitor**.
3. The user can also send commands from the Serial Monitor to Arduino.
4. Arduino reads the incoming data using **Serial.read( )**.

This creates **two-way communication** between the Arduino and the computer.

Basic Serial Communication Program

```
void setup( )
{
  Serial.begin(9600);
}

void loop()
{
  Serial.println("Hello Arduino");
  delay(1000);
}
```

Serial.begin(9600); initializes serial communication with a **baud rate of 9600 bits per second**. Serial.println( ) sends data to the Serial Monitor. The message is displayed every **1 second**.

Sending Data from Arduino to Serial Monitor

The Arduino can transmit different types of data.

```
void setup( )
{
  Serial.begin(9600);
}

void loop( )
{
  int sensorValue = analogRead(A0);

  Serial.print("Sensor value: ");
  Serial.println(sensorValue);

  delay(500);
}
```

The analog sensor value is read from **pin A0**. The value is transmitted to the computer through the serial interface. The Serial Monitor displays the sensor data

Receiving Data from Serial Monitor

Arduino can also receive commands sent from the Serial Monitor.

```
int led = 13;

void setup( )
{
  Serial.begin(9600);
  pinMode(led, OUTPUT);
```

```
}

void loop()
{
  if (Serial.available( ) > 0)
  {
    char data = Serial.read( );

    if(data == '1')
      digitalWrite(led, HIGH);

    if(data == '0')
      digitalWrite(led, LOW);
  }
}
```

Serial.available( ) checks if data is available.
Serial.read( ) reads the incoming character.
Sending **1** from the Serial Monitor turns the LED **ON**.
Sending **0** turns the LED **OFF**.

Important Serial Functions

Function	Purpose
Serial.begin( )	Initializes serial communication
Serial.print( )	Prints data without new line
Serial.println( )	Prints data with new line
Serial.read( )	Reads incoming data
Serial.available( )	Checks for received data

Software Serial

Most Arduino boards have **only one hardware serial port**, which is used for USB communication with the computer. Sometimes we need to communicate with additional devices such as:

- GPS modules
- Bluetooth modules
- Wi-Fi modules
- Other microcontrollers

To solve this problem, Arduino provides the **SoftwareSerial library**.
Software Serial is a method of creating **additional serial ports using digital pins** through software rather than hardware. It allows Arduino to communicate with multiple serial devices simultaneously.

Example

```
#include <SoftwareSerial.h>

SoftwareSerial mySerial(2, 3);

void setup()
{
  Serial.begin(9600);
  mySerial.begin(9600);
}

void loop()
{
  if (mySerial.available())
  {
    char data = mySerial.read();
    Serial.println(data);
  }
}
```

- Pin **2** → RX (receive pin)
- Pin **3** → TX (transmit pin)
- mySerial.begin( ) initializes the software serial communication.
- Data received from the external device is displayed on the Serial Monitor.

Advantages of Software Serial

- Allows **multiple serial devices** to connect to Arduino.
- Useful when hardware serial ports are limited.
- Flexible because any digital pin can be used.

Limitations of Software Serial

- Slower than hardware serial communication.
- Higher CPU usage because it is handled by software.
- Not suitable for **very high data rates**

Two common **uses of the Arduino serial port** are:

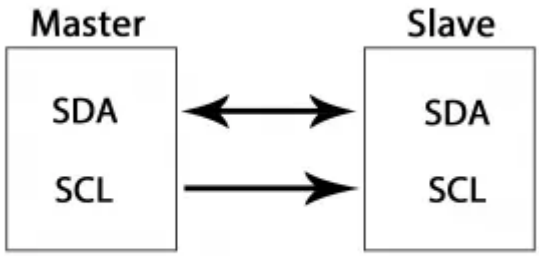
1. **Communication with a Computer (Serial Monitor)**
   The Arduino serial port allows communication between the **Arduino board and a computer** through the USB cable. It is commonly used to **send data from Arduino to the Serial Monitor** or **receive commands from the Serial Monitor**.
   Example: Displaying sensor readings on the Serial Monitor.
2. **Communication with Other Devices**
   The serial port can be used to communicate with **other microcontrollers, sensors, or modules** such as **GPS modules, Bluetooth modules, or GSM modules** using serial communication protocols.

I2C Protocol

I2C stands for "**inter-Integrated Circuit** bus". It is also called as I2C. I2C is a two-wire serial protocol for connecting **low-speed devices** such as microcontrollers, EEPROMs, **Analog to Digital** (A/D) and **Digital to Analog** (D/A) converters, **Input/Output** (I/O) interfaces and other similar peripherals in an **embedded system**.

The I2C bus is famous because it is **easy to use**, it can have more than one master and multiple slaves, only upper bus speed is defined and only two wires with **pull-up resistors** are needed to connect a nearly unlimited number of I2C devices.

I2C requires just 2 wires to transfer data between devices:

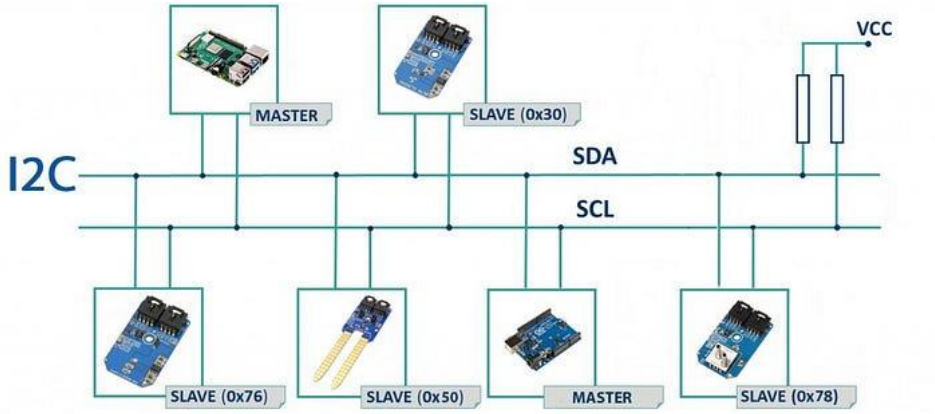


I2C Communication

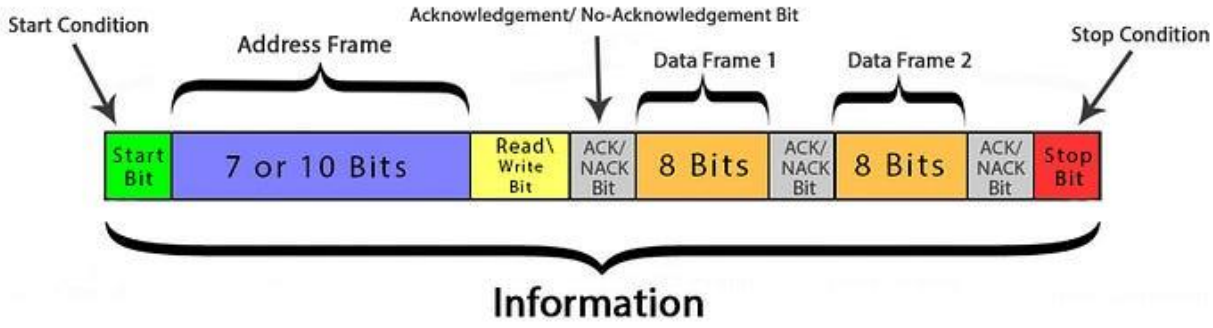
1. **SCL (Serial Clock)**-The clock signal line.
2. **SDA (Serial Data)**-The line for sending and receiving data between master and slave.

*I2C is a **serial communication** protocol and the SDA line is used to transmit the **data bit by bit**.*

Each device is provided preset ID or a unique device address so that the master can select which devices to use. **Serial Clock (or SCL)** and **Serial Data (or SDA)** are the two wires or lines. The **SCL line** is used to **transmit the clock signal** that synchronizes the transfer of data between the devices on the I2C bus, and the master generates the clock signal. The other line is the **SDA line** that is used to **send the data**.



Steps to transfer data between master and slave device:



The communication between master and slave occurs in the following steps:

- **Start Condition**

The master begins communication by sending a **START signal**.

SDA ↓ while SCL is HIGH

- **Address Transmission**

The master sends the **slave address** along with the **read/write bit**.

Each slave device has a **unique address**.

Typical address format:

| 7-bit Address | R/W Bit |

7-bit Address → identifies the slave device

R/W Bit

- 0 → Write operation

- 1 → Read operation

0101000 0 → Write to device

0101000 1 → Read from device

❖ **Acknowledgment (ACK)**

The slave sends an **ACK signal** indicating that it is ready to communicate.

❖ **Data Transfer**

Data is transmitted between master and slave.

- **Master Write Mode**

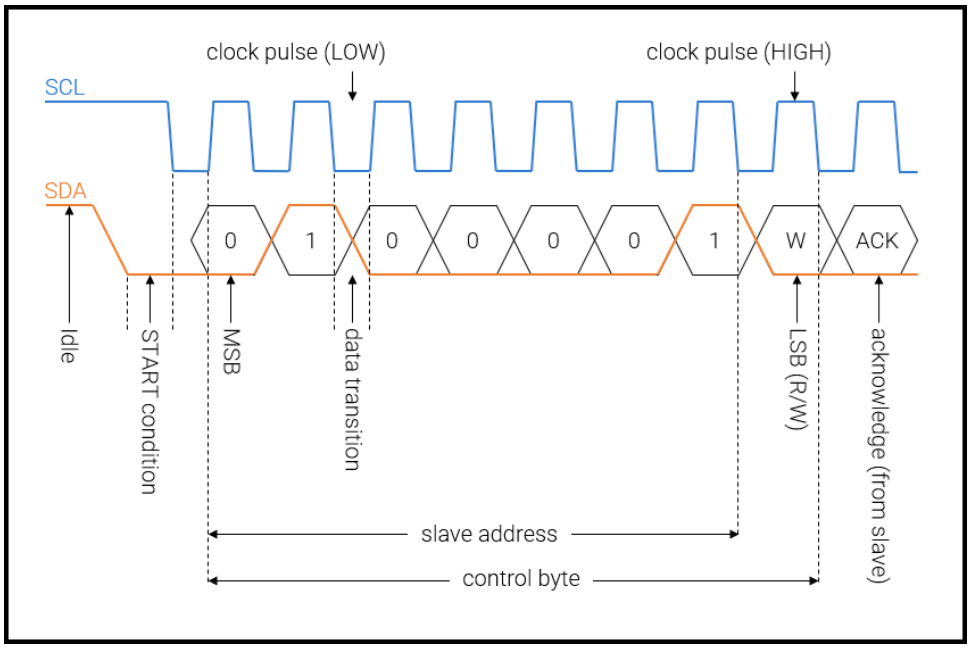
  Master → Slave

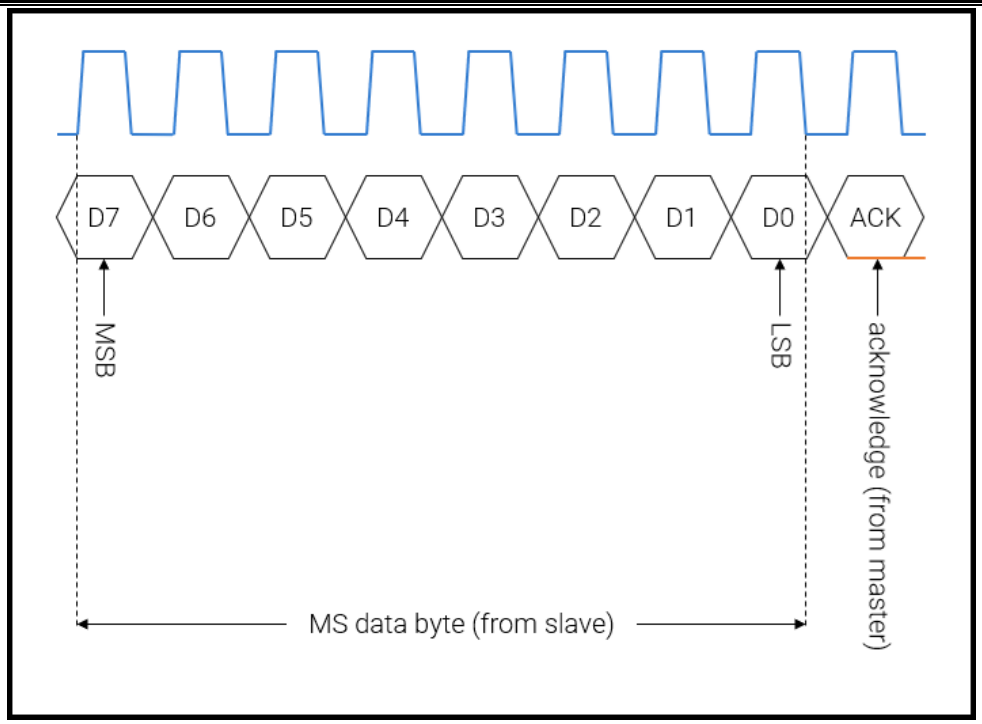
- **Master Read Mode**

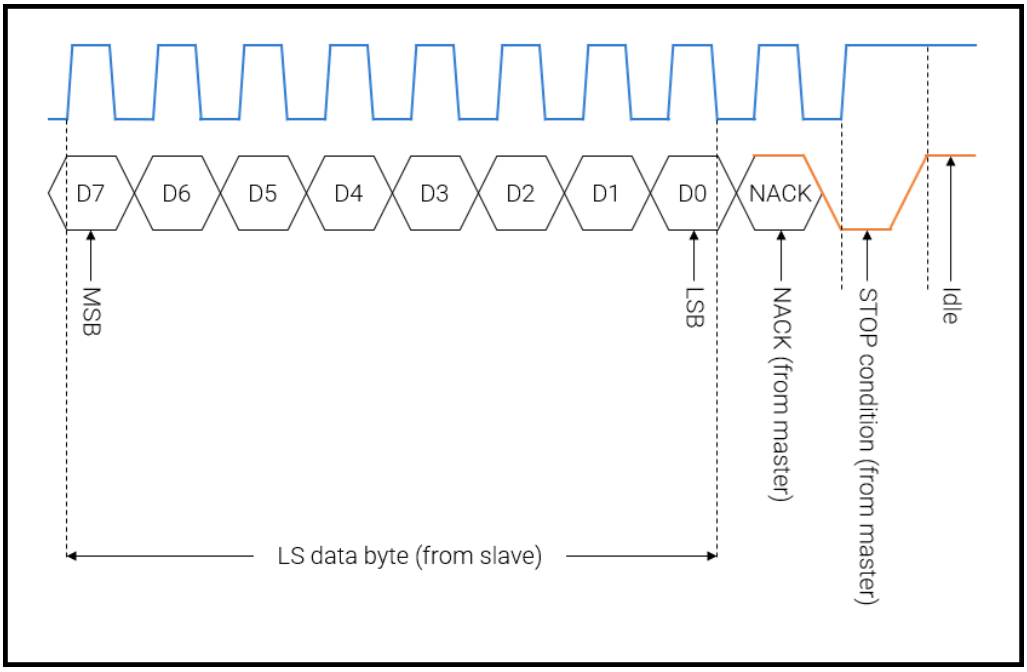
  Slave → Master

❖ **Stop Condition**

Communication ends with a **STOP signal**.



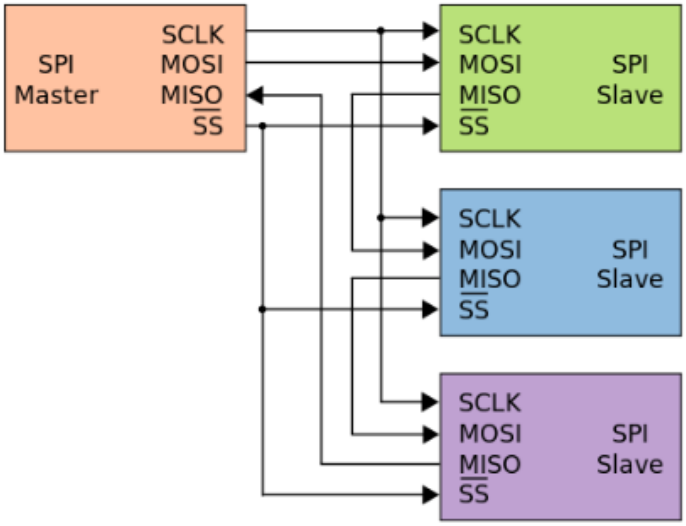




SPI (Serial Peripheral Interface)

SPI (Serial Peripheral Interface) is a **synchronous serial communication protocol** used for high-speed communication between a microcontroller and peripheral devices such as sensors, memory chips, displays, and ADC/DAC converters.

Unlike I2C, SPI uses **separate data lines for transmitting and receiving**, allowing faster communication. SPI communication generally follows a **master–slave architecture**, where the master device controls the communication and the slave devices respond accordingly.



SPI communication uses four primary signal lines, which are dedicated hardware pins on Arduino boards:

- **SCK** (Serial Clock): The clock signal generated by the controller to synchronize data transmission (Arduino Uno Pin 13).
- **MOSI** (Master Out Slave In): The data line from the controller to the peripheral (Arduino Uno Pin 11).
- **MISO** (Master In Slave Out): The data line from the peripheral to the controller (Arduino Uno Pin 12).
- **SS/CS** (Slave Select/Chip Select): An active-low signal used by the controller to enable a specific peripheral device. Each peripheral needs its own dedicated CS pin from the controller

Software Implementation with the SPI Library

The Arduino IDE includes a built-in SPI.h library to manage the communication. Key functions for using the library include:

Function	Purpose
#include <SPI.h>	Includes the SPI library in your sketch.
SPI.begin( )	Initializes the SPI bus by setting the MOSI, MISO, SCK, and SS pins to their proper states.

SPI.beginTransaction(settings)	Configures the clock speed, data order, and SPI mode for a specific device using a SPISettings object.
SPI.transfer(value)	Simultaneously sends a byte of data on the MOSI line and receives a byte of data on the MISO line.
SPI.endTransaction( )	Ends the SPI transaction, allowing other devices that might use different settings to use the bus.

Typical Communication Sequence

1. **Start Transaction**: The controller calls SPI.beginTransaction( ) to configure settings specific to the target peripheral.
2. **Select Peripheral**: The controller pulls the target peripheral's **CS pin LOW** to enable it.
3. **Transfer Data**: The controller calls SPI.transfer( ) to exchange data one byte at a time, synchronized by the SCK signal.
4. **Deselect Peripheral**: Once the data exchange is complete, the controller pulls the **CS pin HIGH** to disable the peripheral.
5. **End Transaction**: The controller calls SPI.endTransaction( ) to release the SPI bus.

Advantages of SPI

- **High-speed communication**
- **Full-duplex data transfer**
- **Simple protocol**
- **No addressing required**
- **Efficient for short-distance communication**

Limitations of SPI

- Requires **more wires**
- Each slave needs a **separate Slave Select pin**
- No built-in **acknowledgement system**
- Not suitable for very large networks

COMPARING SPI WITH I2C AND UART

SPI	I²C	UART
Can operate at the highest speeds.	Maximum speed is highly dependent upon physical characteristics of the bus (length, number of devices, pull-up strength, and so on).	Requires baud rate to be agreed upon by both devices in advance of starting communication.
Is generally easier to work with than I²C.	Requires only two communication lines.	Effectively uses zero protocol overhead—very simple to implement.
No pull-up resistors needed.	Can support communication between devices operating from different voltage rails.	No predefined master/slave—it is up to you to define the protocol.
Number of slave devices is limited only by number of available SS pins on master.	Number of slave devices is limited by availability of chips with particular slave addresses.	Cannot easily support multiple slave devices.
Has built-in Arduino hardware and software support.	Has built-in Arduino hardware and software support.	Has built-in Arduino hardware and software support.

Communicating with an SPI Accelerometer

An **accelerometer** is a sensor that measures **acceleration, tilt, vibration, and motion** along one or more axes (X, Y, Z). Many modern accelerometers provide digital outputs and communicate with microcontrollers using protocols such as **SPI or I2C**.

Using **SPI communication**, the Arduino can read acceleration data quickly and efficiently because SPI provides **high-speed full-duplex communication.**

When connecting an SPI accelerometer to Arduino, four SPI signals are required.

SPI Pin	Function
MOSI	Data from Arduino to sensor
MISO	Data from sensor to Arduino
SCK	Clock signal generated by Arduino
CS / SS	Chip Select to enable the accelerometer

When **CS is LOW**, the accelerometer is active. When **CS is HIGH**, communication is disabled.

SPI accelerometers internally contain **MEMS sensing elements** that detect motion along three axes:

- **X-axis**
- **Y-axis**
- **Z-axis**

The sensor converts physical acceleration into **digital values stored in internal registers**. The Arduino reads these registers using SPI communication.

Communication Scheme

The SPI communication with an accelerometer follows a **register-based communication scheme**.

Step-by-Step Communication

1. **Enable the sensor**
   - Master pulls **Chip Select (CS) LOW**.
2. **Send register address**
   - Arduino sends the register address through **MOSI**.
3. **Read or write command**
   - A bit in the address indicates whether it is a **read or write operation**.
4. **Clock synchronization**
   - Data transfer occurs synchronized with the **SCK clock pulses**.
5. **Receive data**
   - Sensor sends the data through **MISO**.
6. **End communication**
   - Arduino sets **CS HIGH**.

Code

```
#include <SPI.h>

const int CS = 10; //Pin 10 is commonly used as the Slave Select (SS) pin in Arduino.

void setup( )
{
 Serial.begin(9600);
```

```
  SPI.begin();

  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);
}

byte readRegister(byte reg)
{
  digitalWrite(CS, LOW);

  SPI.transfer(reg | 0x80);   // Read command
  byte value = SPI.transfer(0x00);

  digitalWrite(CS, HIGH);

  return value;
}

void loop( )
{
  byte x = readRegister(0x32);   // X-axis register

  Serial.print("X-axis: ");
  Serial.println(x);

  delay(500);
}
```